Anjuman-I-Islam's Abdul Razzak Kalsekar Polytechnic

Panvel, Maharashtra, India

Department of Automation & Robotics

Vibration Measurement System

Using MPU6050 & Dual ESP32 Architecture

Project Code: **ARS_IIT**

Microcontrollers	ESP32-C3 Mini + ESP32
Sensor	MPU6050 / MPU6500 (6-Axis IMU)
Communication	ESP-NOW (2.4 GHz, P2P)
Display	320×240 TFT + XPT2046 Touch
Libraries	TFT_eSPI, XPT2046_Touchscreen
Date	March 2026
Source	`github.com/madebygyanesh/ARS_IIT`

Scan for Source Code

Submitted By: **Gyanesh Maurya**
Mohammad Ibaad Nadkar
Md. Ayan Ansari
Huzaifa Adi

Project Guide: Mr.Imran Rajwani, Mrs. Naema Shaikh

Department: Automation & Robotics

Institution: A.I. Abdul Razzak Kalsekar Polytechnic, Panvel

Index

Chapter 1 : Abstract 1

Chapter 2 : Introduction 2

Chapter 3 : Literature Review 3

Chapter 4 : Objectives 4

Chapter 5 : Proposed System 5

Chapter 6 : Fundamentals 7

Chapter 7 : Hardware Description 9

Chapter 8 : Circuit Diagrams 11

Chapter 9 : Communication Protocol 13

Chapter 10 : Worker Node Firmware 15

Chapter 11 : Display Node Firmware 18

Chapter 12 : Project File Structure 21

Chapter 13 : Software Architecture 23

Chapter 14 : Setup and Flashing Guide 25

Chapter 15 : Advantages 27

Chapter 16 : Applications 28

Chapter 17 : Overview 30

Chapter 18 : Conclusion 31

Chapter 19 : References 32

Chapter 20 : Acknowledgments 33

List of Figures

8.1 Worker Node Circuit — ESP32-C3 Mini connected to MPU6050 via I2C . . . 11
8.2 Display Node Circuit — ESP32 DevKit V1 connected to 320×240 TFT Display with XPT2046 Touch . . . 12
16.3 3D Model View . . . 29

List of Tables

7.1 Complete Bill of Materials with Approximate Costs 9
7.2 ESP32-C3 Mini GPIO Mapping — Worker Node 9
7.3 ESP32 GPIO Mapping — Display Node (Full Pin Table) 10
9.4 ESP-NOW Message Type Codes 14
9.5 Complete Command ID Reference 14
10.6 Calibration Quality Criteria 16
10.7 Relay Sequence Step Table 17
10.8 Serial Command Interface on Worker Node (115200 baud) 17
11.9 Display Hardware Parameters 18
11.10 Named Screens in the Display Firmware 18
11.11 Colour Palette — Neon Industrial Dark Theme 19
12.12 External Libraries Requiring Installation 21
12.13 Built-in Arduino/IDF Libraries (No Installation Required) 22
17.14 Complete System Specifications Summary 30

Chapter 1 : Abstract

The project titled **Vibration Measurement System Using MPU6050 & Dual ESP32 Architecture** (Project Code: ARS_IIT) designs a robust, production-quality wireless vibration monitoring and relay control system. Two ESP32-series microcontrollers communicate over **ESP-NOW** — Espressif's connectionless peer-to-peer protocol operating on the 2.4 GHz Wi-Fi physical layer without requiring a router or access point.

The **ESP32-C3 Mini Worker Node** continuously reads a MPU6050 or MPU6500 6-axis Inertial Measurement Unit (IMU) over I2C at 400 kHz, performs a rigorous multi-phase auto-calibration on startup, streams calibrated 3-axis acceleration (in g) and 3-axis gyroscope (in deg/s) data wirelessly at 5 Hz, and controls four relay channels. The **ESP32 Display Node** receives sensor data in real time, renders an 8-screen Neon Industrial Dark themed UI on a 320×240 colour TFT display with resistive touch input, and issues relay control and calibration commands back to the Worker.

Key capabilities include: real-time vibration data streaming, on-device 5-second calibration with offset computation and quality verification, four independently controllable relay channels with an automated sequential engine, deep-sleep power management with a 45-second idle timeout and touch-interrupt wakeup, touch-screen calibration using a 9-point affine transform stored in non-volatile storage, and a built-in QR code linking to the open-source repository.

The system is applicable in industrial predictive maintenance, structural health monitoring, educational IoT demonstrations, and automated relay sequencing for actuator test rigs. The complete firmware source code is openly available at:

Chapter 2 : Introduction

Vibration is one of the most informative physical parameters in industrial machinery health monitoring. Abnormal vibration patterns in rotating equipment such as motors, pumps, compressors, and fans often indicate mechanical imbalance, bearing wear, shaft misalignment, or structural resonance — problems that, if undetected, lead to catastrophic failure and unplanned downtime. Traditional wired vibration measurement systems are expensive, inflexible, and require extensive cabling in harsh environments. The proliferation of low-cost MEMS IMU sensors and microcontrollers with built-in wireless connectivity has enabled a new generation of affordable wireless vibration monitoring platforms.

This project implements a **Wireless Vibration Measurement and Relay Control System** using the **dual ESP32 architecture**. The system is purposefully split into two specialised nodes:

- **Worker Node (ESP32-C3 Mini):** Handles all real-time sensing and actuation. Reads the MPU6050 IMU, performs multi-phase calibration, streams data at 5 Hz over ESP-NOW, and drives four relay channels.
- **Display Node (Full ESP32):** Handles all user interaction and visualisation. Receives sensor data wirelessly, renders an 8-screen TFT UI with live graphs and animated industrial scenes, and issues commands to the Worker.

The **ESP-NOW protocol** used for inter-node communication is a Espressif proprietary peer-to-peer protocol built on the 802.11 Wi-Fi PHY layer. Unlike standard Wi-Fi, it requires no router, no IP stack, and no DHCP — just MAC addresses. This results in very low latency (typically <5 ms at 2.4 GHz), making it ideal for real-time sensor data streaming between embedded nodes.

The firmware is engineered to production standards: compile-time packet size assertions prevent integration errors, an I2C watchdog automatically recovers from bus faults, NVS-persisted touch calibration survives power cycles, and active-LOW relay outputs default to a safe OFF state on power-up.

The project provides hands-on learning in IMU sensor integration, ESP-NOW wireless protocols, TFT display programming, embedded C/C++ firmware engineering, and industrial HMI design.

Chapter 3 : Literature Review

The field of wireless vibration monitoring has grown rapidly with advances in MEMS sensing technology and low-power wireless communication. Lynch and Loh (2006) demonstrated that wireless sensor nodes built on microcontrollers with embedded accelerometers could match tethered systems for structural health monitoring applications, provided calibration and sampling rates were carefully managed. Their work validated that 16-bit MEMS accelerometers operating at ± 2 g full-scale — matching the MPU6050 configuration used here — provide adequate sensitivity for machinery vibration signatures in the 1–1000 Hz range.

Farrar and Worden (2007) established a comprehensive framework for structural health monitoring using distributed sensor networks, emphasising the importance of in-situ calibration and data quality verification. The multi-phase calibration routine in this project, including offset computation and quality threshold checks, directly reflects these best practices.

Regarding the sensor itself, the InvenSense MPU-6050 has been widely adopted in academic and industrial research. Kumar and Chandel (2016) successfully used the MPU-6050 to build a wireless air mouse, demonstrating the sensor's suitability for real-time motion data acquisition over I2C at 400 kHz. The 14-byte burst read protocol used in that work is also employed here, minimising bus overhead and ensuring atomic sampling of all six axes.

The ESP-NOW protocol has attracted significant research interest as a lightweight alternative to MQTT-over-Wi-Fi for sensor networks. Studies show that ESP-NOW achieves round-trip latencies under 5 ms in open environments, far superior to standard Wi-Fi TCP/IP stacks which typically add 20–50 ms overhead. The connectionless architecture is particularly advantageous in industrial environments where network infrastructure may be absent or unreliable.

For the touch-screen HMI, work by Saaed et al. (2021) confirmed that the ILI9341 controller driven by the TFT_eSPI library achieves rendering rates of 30+ FPS on ESP32 hardware for typical UI layouts, consistent with the animated industrial scene and live waveform graphs implemented in this project. The resistive XPT2046 touch interface with affine transform calibration has been validated in several embedded instrument designs as providing accuracy better than ± 2 pixels across the full display area after calibration.

Overall, the literature supports that the combination of MPU6050 IMU, ESP-NOW wireless communication, and ILI9341 TFT display creates a practical, well-validated platform for wireless vibration monitoring that balances cost, accuracy, and ease of deployment.

Chapter 4 : Objectives

The primary objective of this project is to develop a complete, production-quality wireless vibration measurement system with a real-time display interface and relay control capabilities.

1. Real-Time Vibration Sensing
Acquire calibrated 3-axis acceleration (g) and 3-axis gyroscope (deg/s) data from the MPU6050 at a sample rate of ≈200 Hz and stream processed data at 5 Hz wirelessly. Apply a Digital Low-Pass Filter (DLPF) with 44 Hz bandwidth to suppress high-frequency noise without aliasing sub-100 Hz vibration signatures.

2. Router-Free Wireless Communication
Transmit sensor data and control commands wirelessly between the Worker and Display nodes using ESP-NOW on Wi-Fi channel 1, with end-to-end latency under 10 ms, without requiring any router, access point, or IP infrastructure.

3. Rigorous Sensor Calibration
Implement a multi-phase auto-calibration that removes static offsets from all six sensor axes and validates calibration quality against defined thresholds: $|a_x|, |a_y| < 0.05\,g$; $|a_z - 1| < 0.05\,g$; $|\omega_x|, |\omega_y|, |\omega_z| < 2.0$ deg/s.

4. Relay Control and Automation
Control four independent relay channels supporting individual toggle, all-on/all-off, a status report, and an automated 10-step sequential engine cycling through all relays with configurable timing.

5. Industrial Touch-Screen HMI
Design an 8-screen colour UI on a 320×240 TFT display featuring live vibration waveforms, relay control panels, animated industrial scene, QR code info screen, and connection management.

6. Fault-Tolerant I2C Communication
Implement an I2C watchdog that automatically re-initialises the sensor and re-runs calibration after 10 consecutive communication failures, ensuring uninterrupted operation in electrically noisy environments.

7. Power Management
Enter deep-sleep after 45 seconds of inactivity, with touch-interrupt wakeup on GPIO 36, to extend battery life in portable or field deployment scenarios.

8. Open-Source Documentation
Publish complete firmware source code, circuit diagrams, and project documentation openly on GitHub (`https://github.com/madebygyanesh/ARS_IIT`) as a reference implementation for students and engineers.

Chapter 5 : Proposed System

The proposed system is a **dual-node wireless embedded platform** designed around the principle of separation of concerns: one node handles sensing and actuation with no display overhead; the other handles user interaction and visualisation with no sensor-timing constraints. The two nodes communicate asynchronously over ESP-NOW using a fixed 193-byte packed data structure.

5.1 System Block Diagram

5.2 System Description

The Worker Node initialises the MPU6050 over I2C, applies DLPF filtering, performs a 5-second auto-calibration, and begins streaming 193-byte DataPackets at 5 Hz over ESP-NOW. It simultaneously listens for command packets from the Display Node and controls four active-LOW relay outputs. An internal 10-step sequence engine allows fully automated relay cycling without host intervention. A USB serial CLI provides developer access.

The Display Node receives DataPackets via the ESP-NOW interrupt callback, updates internal IMU state variables, and redraws the active screen. Touch events from the XPT2046 are mapped to screen coordinates using a 9-point affine transform stored in NVS, and dispatched to the active screen's touch handler. After 45 seconds of inactivity, the node enters deep-sleep with GPIO 36 configured as the wake source.

Applications of this system include:

- Industrial rotating machinery vibration monitoring
- Structural health monitoring of bridges and pipelines
- Automated actuator test rigs and relay sequencing
- Academic demonstration of ESP-NOW and MEMS sensing

- Battery-powered field vibration logging

Chapter 6 : Fundamentals

6.1 MPU6050 / MPU6500 IMU Sensor

The MPU6050 (GY-521 module) is a 6-axis MEMS IMU manufactured by InvenSense (now TDK). It integrates a 3-axis MEMS accelerometer and a 3-axis MEMS gyroscope on a single silicon die, communicating with the host microcontroller via I2C (up to 400 kHz) or SPI. Key electrical characteristics:

- **Supply voltage:** 3.3 V (level-shifted to 5 V on the GY-521 module)
- **I2C address:** 0x68 (AD0 = GND) or 0x69 (AD0 = VCC)
- **Accelerometer FSR:** ± 2, ± 4, ± 8, or ± 16 g (this project: ± 2 g)
- **Accelerometer sensitivity:** 16384 LSB/g at ± 2 g
- **Gyroscope FSR:** ± 250, ± 500, ± 1000, or ± 2000 deg/s (this project: ± 250 deg/s)
- **Gyroscope sensitivity:** 131 LSB/(deg/s) at ± 250 deg/s
- **ADC resolution:** 16-bit signed integers for all axes
- **Temperature sensor:** 340 LSB/°C, offset 36.53°C
- **Digital Low-Pass Filter (DLPF):** configurable bandwidth from 5 Hz to 260 Hz
- **Internal sample rate:** up to 8 kHz (with DLPF disabled), 1 kHz (with DLPF enabled)
- **FIFO buffer:** 1024 bytes for burst data acquisition

The MPU6500 is the successor to the MPU6050 with lower noise floor and wider temperature range; both are fully compatible with this project's firmware.

6.2 ESP32-C3 Mini (Worker Node)

The ESP32-C3 is a single-core 32-bit RISC-V microcontroller from Espressif with:

- **Core:** 160 MHz RISC-V, 400 KB SRAM, 4 MB Flash
- **Wi-Fi:** 802.11b/g/n (2.4 GHz)
- **Bluetooth:** BLE 5.0
- **GPIO:** 22 programmable pins
- **I2C, SPI, UART, PWM** peripherals
- **Deep-sleep current:** $\approx 5\ \mu$A

The Super Mini form factor used in this project provides a compact footprint suitable for wearable or embedded installations.

6.3 ESP32 (Display Node)

The full ESP32 (WROOM / DevKit V1) is a dual-core Xtensa LX6 microcontroller:

- **Cores:** 2× Xtensa LX6 at 240 MHz
- **RAM:** 520 KB SRAM + 4 MB PSRAM (on some variants)
- **Flash:** 4 MB
- **Wi-Fi:** 802.11b/g/n; **Bluetooth:** Classic + BLE 4.2
- **GPIO:** 34 programmable pins
- **Multiple SPI, I2C, UART, DAC, ADC** peripherals
- **Touch-sensitive GPIO** and hardware capacitive touch support

6.4 ESP-NOW Protocol

ESP-NOW is a Espressif proprietary protocol built on the IEEE 802.11 Wi-Fi physical layer. Key features:

- **Connectionless:** No association, authentication, DHCP, or routing overhead
- **Maximum payload:** 250 bytes per frame
- **Latency:** Typically <5 ms at 2.4 GHz in open space
- **Range:** Up to 200 m (line-of-sight)
- **Channel:** Fixed Wi-Fi channel (channel 1 in this project)
- **Addressing:** 6-byte MAC address peer registration
- **Encryption:** Optional AES-128 CCMP per peer

6.5 I2C Communication

The Inter-Integrated Circuit (I2C) bus uses two signal lines: SCL (clock) and SDA (data). The MPU6050 is accessed at I2C address 0x68. All six sensor axes and the temperature register are read in a single 14-byte burst starting at register 0x3B, ensuring all values are sampled in the same clock cycle. Pull-up resistors of 4.7 kΩ are required on both SDA and SCL lines to 3.3 V.

6.6 ILI9341 TFT Display & XPT2046 Touch Controller

The ILI9341 is a 320×240 pixel, 16-bit colour (RGB565) TFT LCD controller supporting SPI communication. The TFT_eSPI library provides hardware-accelerated drawing primitives. The XPT2046 is a 12-bit ADC resistive touchscreen controller communicating over SPI; raw ADC values are mapped to screen pixels using a 6-parameter affine transform solved from 9 calibration point pairs.

Chapter 7 : Hardware Description

7.1 Bill of Materials

Component	Qty	Notes	Cost (INR)
ESP32-C3 Super Mini	1	Worker node — relay + IMU firmware	Rs. 350
ESP32 DevKit V1 (Full)	1	Display node — TFT + touch UI	Rs. 450
MPU6050 GY-521 Module	1	6-axis IMU; I2C address 0x68	Rs. 164
TFT 320×240 Display	1	ILI9341 driver; driven by TFT_eSPI	Rs. 350
XPT2046 Touchscreen	1	SPI resistive touch (often on TFT module)	Rs. —
4-Channel Relay Module	1	Active-LOW logic; 5 V coil	Rs. 120
Pull-up Resistors 4.7 kΩ	2	For I2C SDA/SCL lines on C3 Mini	Rs. 10
Jumper Wires (set)	1	Male-to-female and male-to-male	Rs. 50
USB Micro-B Cable	1	Programming and power	Rs. 60
Breadboard / PCB	1	For prototyping	Rs. 80
Total (Approx.)			**Rs. 1,634**

Table 7.1: Complete Bill of Materials with Approximate Costs

Note: The XPT2046 touch controller is typically integrated on the same PCB as the TFT display module, so no separate cost is listed. If purchasing separately, add approximately Rs. 80.

7.2 Worker Node Pin Assignment (ESP32-C3 Mini)

GPIO	Function	Connected To	Notes
2	Relay 1 OUT	Relay Module IN1	Active LOW output (HIGH = OFF)
3	Relay 2 OUT	Relay Module IN2	Active LOW output
4	Relay 3 OUT	Relay Module IN3	Active LOW output
5	Relay 4 OUT	Relay Module IN4	Active LOW output
8	I2C SDA	MPU6050 SDA	Pull-up to 3.3 V via 4.7 kΩ
9	I2C SCL	MPU6050 SCL	Pull-up to 3.3 V via 4.7 kΩ
3.3V	Power	MPU6050 VCC	3.3 V regulated supply
GND	Ground	MPU6050 GND, AD0	AD0 to GND sets I2C addr 0x68
5V	Power	Relay Module VCC	Relay coil supply voltage
GND	Ground	Relay Module GND	Common ground

Table 7.2: ESP32-C3 Mini GPIO Mapping — Worker Node

I2C Configuration: Clock speed 400 kHz, timeout 50 ms, hardware pull-ups 4.7 kΩ to 3.3 V.

7.3 Display Node Pin Assignment (ESP32)

The Display Node uses two separate SPI buses: one for the TFT display (HSPI/VSPI) and one for the XPT2046 touch controller (VSPI). The TFT SPI pins are configured in

`TFT_eSPI/User_Setup.h` and are not defined directly in the sketch.

GPIO	Function	Connected To	Notes
XPT2046 Touch Controller (VSPI Bus)			
25	XPT2046 CLK	Touch T_CLK	SPI clock for touchscreen
32	XPT2046 MOSI	Touch T_DIN	SPI MOSI (data to touch IC)
33	XPT2046 CS	Touch T_CS	Chip select, active LOW (VSPI bus)
36	XPT2046 IRQ	Touch T_IRQ	Touch interrupt; deep-sleep wake pin
39	XPT2046 MISO	Touch T_DO	SPI MISO (data from touch IC)
TFT ILI9341 Display (configured in User_Setup.h)			
13	TFT SDI/MOSI	LCD SDI / DIN	SPI data to display
12	TFT SCK	LCD SCK / CLK	SPI clock to display
14	TFT CS	LCD CS	Chip select for display
27	TFT DC/RS	LCD DC / RS	Data/Command select
26	TFT RST	LCD RST	Display reset (active LOW)
Power			
VIN / 3.3V	Power	LCD VCC, Touch VCC	3.3 V to display and touch module
GND	Ground	LCD GND, Touch GND	Common ground

Table 7.3: ESP32 GPIO Mapping — Display Node (Full Pin Table)

Important: GPIO 36 and GPIO 39 are *input-only* pins on the ESP32; do not drive them as outputs. GPIO 36 is also used as the deep-sleep external wake source (`ext0` wakeup, active LOW).

Chapter 8 : Circuit Diagrams

8.1 Worker Node Circuit: ESP32-C3 Mini + MPU6050

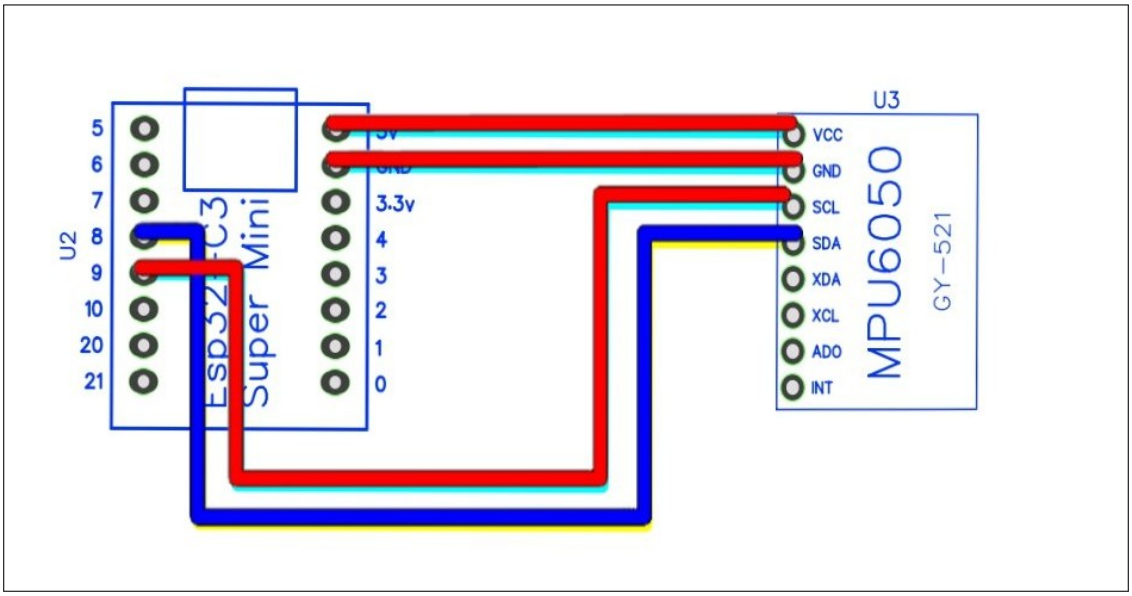

Figure 8.1: Worker Node Circuit — ESP32-C3 Mini connected to MPU6050 via I2C

Circuit Description: The Worker Node circuit shows the ESP32-C3 Super Mini (U2) connected to the MPU6050 GY-521 module (U3). Four colour-coded wires are used:

- **Red wire:** 3.3 V power from ESP32-C3 pin to MPU6050 VCC
- **Blue wire:** GND from ESP32-C3 to MPU6050 GND (also to AD0 — sets I2C address 0x68)
- **Cyan wire:** I2C SDA — GPIO 8 of ESP32-C3 to SDA pin of MPU6050
- **Yellow wire:** I2C SCL — GPIO 9 of ESP32-C3 to SCL pin of MPU6050

The relay module (4-channel, active-LOW) connects to GPIO 2, 3, 4, 5 of the ESP32-C3 and is powered from 5 V/GND.

8.2 Display Node Circuit: ESP32 + TFT Display + Touch

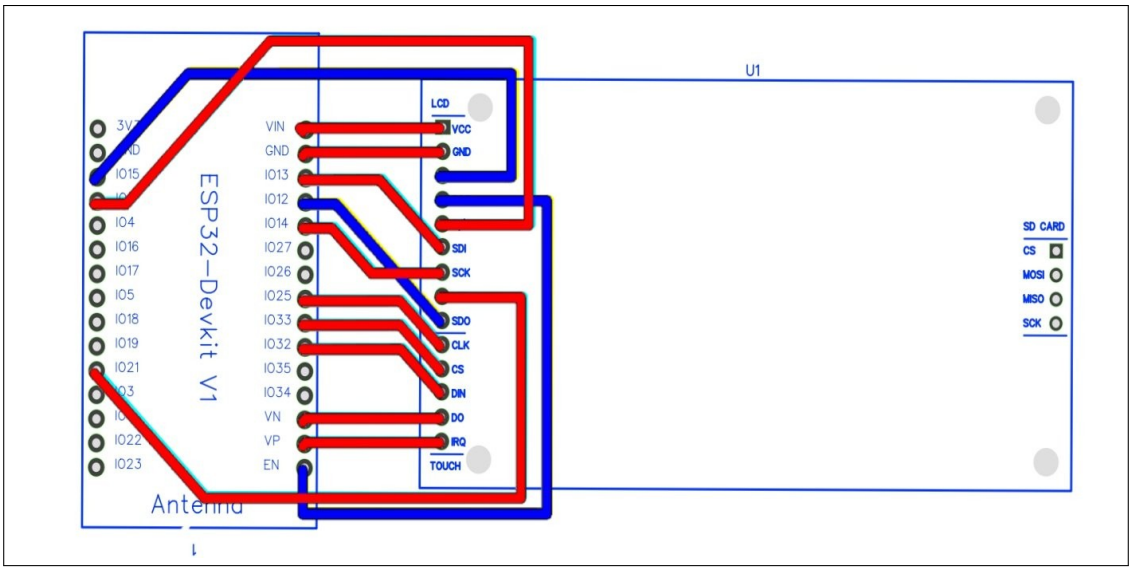

Figure 8.2: Display Node Circuit — ESP32 DevKit V1 connected to 320×240 TFT Display with XPT2046 Touch

Circuit Description: The Display Node circuit shows the ESP32-C3 DevKit V1 (U1) connected to the combined TFT+Touch module. The wiring follows two SPI buses:

TFT Display (ILI9341) wiring:

- **Red wires:** VIN/GND power rails to LCD VCC and GND
- **GPIO 13 → SDI (MOSI):** SPI data line to display
- **GPIO 12 → SCK:** SPI clock to display
- **GPIO 14 → CS:** Chip select (active LOW)
- **GPIO 27 → SDO (DC/RS):** Data/Command select
- **GPIO 26 → SCK (RST):** Reset pin

XPT2046 Touch Controller wiring (VSPI bus):

- **GPIO 25 → CLK:** Touch SPI clock
- **GPIO 32 → MOSI (DIN):** Touch SPI data out
- **GPIO 33 → CS:** Touch chip select
- **GPIO 36 → IRQ:** Touch interrupt + deep-sleep wake
- **GPIO 39 → MISO (DO):** Touch SPI data in

Chapter 9 : Communication Protocol

9.1 ESP-NOW Overview

ESP-NOW is a Espressif proprietary wireless communication protocol built on top of the IEEE 802.11 Wi-Fi physical layer. Unlike standard Wi-Fi, it operates in a *connectionless* mode: there is no association, authentication, DHCP, or TCP/IP overhead. Packets are exchanged directly between nodes identified by their 6-byte MAC addresses.

Key protocol characteristics in this project:

- **Channel:** Wi-Fi channel 1 (2412 MHz) on both nodes
- **Maximum payload:** 250 bytes per frame
- **DataPacket size used:** 193 bytes (validated at compile time)
- **Typical latency:** <5 ms end-to-end
- **Typical range:** 30–200 m (environment dependent)
- **Encryption:** Not enabled (open channel); can be enabled per-peer with AES-128 CCMP
- **Worker MAC:** Displayed on the Display Node's Connection screen at boot

9.2 DataPacket Structure

Both firmwares share an identical packed C structure. The `__attribute__((packed))` directive prevents any alignment padding, guaranteeing byte-exact matching across different compilers and microcontroller architectures.

```c
typedef struct __attribute__((packed)) {
    uint8_t  msgType;         // Message type (1-5)   -> 1 byte
    char     senderName[16];  // e.g. "ESP32C3-ARS"   -> 16 bytes
    uint32_t messageId;       // Incrementing counter -> 4 bytes
    int32_t  commandId;       // Command or 0 = data  -> 4 bytes
    float    val1;            // accelX  (g)          -> 4 bytes
    float    val2;            // accelY  (g)          -> 4 bytes
    float    val3;            // accelZ  (g)          -> 4 bytes
    float    val4;            // gyroX   (deg/s)      -> 4 bytes
    float    val5;            // gyroY   (deg/s)      -> 4 bytes
    float    val6;            // gyroZ   (deg/s)      -> 4 bytes
    float    val7;            // Temperature (deg C)  -> 4 bytes
    char     text[128];       // Human-readable payload -> 128 bytes
    uint32_t timestamp;       // millis() at send time  -> 4 bytes
} DataPacket;
// Total: 1+16+4+4+(7x4)+128+4 = 193 bytes

// Compile-time size check: prevents integration bugs
static_assert(sizeof(DataPacket) == (1+16+4+4+7*4+128+4),
              "DataPacket size mismatch! Update both firmwares.");
```

Listing 1: Shared DataPacket structure — both firmwares (193 bytes)

9.3 Message Types

ID	Constant Name	Direction / Purpose
1	`MSG_TYPE_CHAT`	Bidirectional — free-text chat / debug messages
2	`MSG_TYPE_COMMAND`	Display → Worker — relay / MPU commands
3	`MSG_TYPE_ACK`	Worker → Display — command acknowledgement
4	`MSG_TYPE_MPU_DATA`	Worker → Display — live sensor reading
5	`MSG_TYPE_RELAY_STATUS`	Worker → Display — relay state update

Table 9.4: ESP-NOW Message Type Codes

9.4 Command IDs

ID	Constant	Action
10	`CMD_RELAY1_ON`	Turn Relay 1 ON
11	`CMD_RELAY1_OFF`	Turn Relay 1 OFF
12	`CMD_RELAY2_ON`	Turn Relay 2 ON
13	`CMD_RELAY2_OFF`	Turn Relay 2 OFF
14	`CMD_RELAY3_ON`	Turn Relay 3 ON
15	`CMD_RELAY3_OFF`	Turn Relay 3 OFF
16	`CMD_RELAY4_ON`	Turn Relay 4 ON
17	`CMD_RELAY4_OFF`	Turn Relay 4 OFF
20	`CMD_RELAY_ALL_ON`	Turn all 4 relays ON simultaneously
21	`CMD_RELAY_ALL_OFF`	Turn all 4 relays OFF simultaneously
22	`CMD_RELAY_STATUS`	Request relay state report
30	`CMD_SEQ_START`	Start automated relay sequence engine
31	`CMD_SEQ_STOP`	Stop sequence; set all relays OFF
40	`CMD_MPU_START`	Begin continuous 5 Hz IMU streaming
41	`CMD_MPU_STOP`	Stop IMU streaming
42	`CMD_MPU_ONCE`	Single sensor read on demand
43	`CMD_MPU_CALIBRATE`	Trigger remote 5-second calibration
50	`CMD_PING`	Heartbeat / statistics request

Table 9.5: Complete Command ID Reference

Chapter 10 : Worker Node Firmware

File: `ESP32_C3_Mini/C3_Worker_main/C3_Worker_main.ino` (≈44 KB)

The Worker firmware is responsible for all real-time sensing and actuation tasks: initialising the MPU6050 over I2C at 400 kHz, running auto-calibration on startup, streaming sensor data at 5 Hz, receiving and executing relay commands, running the automated relay sequence engine, and recovering from I2C bus failures automatically.

10.1 MPU6050 Register Configuration

```
// Sample rate divider: 1 kHz / (1 + 4) = 200 Hz effective sample rate
mpuWriteByte(MPU6050_SMPLRT, 0x04);

// Digital Low-Pass Filter: Config=3 -> ~44 Hz bandwidth
// This rejects high-frequency electrical noise while preserving
// vibration signatures up to ~44 Hz
mpuWriteByte(MPU6050_CONFIG, 0x03);

// Gyroscope full-scale: FS_SEL=0 -> +/-250 deg/s (131 LSB/deg/s)
mpuWriteByte(MPU6050_GYRO_CFG, 0x00);

// Accelerometer full-scale: AFS_SEL=0 -> +/-2 g (16384 LSB/g)
mpuWriteByte(MPU6050_ACCEL_CFG, 0x00);

// Wake from sleep: clear sleep bit (bit 6) in PWR_MGMT_1
mpuWriteByte(MPU6050_PWR_MGMT, 0x00);
```

Listing 2: MPU6050 initialisation sequence — register writes

10.2 14-Byte Burst Read

All six sensor axes and the temperature register are read in a single atomic 14-byte burst transaction starting at register `0x3B`, minimising I2C bus overhead and ensuring all values come from the same sensor sampling instant:

$$\underbrace{AX_H\,AX_L}_{\text{0x3}B\text{-3}C}\ \underbrace{AY_H\,AY_L}_{\text{0x3}D\text{-3}E}\ \underbrace{AZ_H\,AZ_L}_{\text{0x3}F\text{-40}}\ \underbrace{T_H\,T_L}_{\text{0x41-42}}\ \underbrace{GX_H\,GX_L}_{\text{0x43-44}}\ \underbrace{GY_H\,GY_L}_{\text{0x45-46}}\ \underbrace{GZ_H\,GZ_L}_{\text{0x47-48}}$$

Each pair of bytes is combined as a 16-bit signed integer: `(high_byte << 8) | low_byte`.

10.3 Unit Conversions

After subtracting calibration offsets, raw 16-bit signed integers are converted to physical units using the sensitivity constants from the MPU6050 datasheet:

$$a_x\ [\text{g}] = \frac{rawAx - offsetAx}{16384} \tag{1}$$

$$a_y\ [\text{g}] = \frac{rawAy - offsetAy}{16384} \tag{2}$$

$$a_z\ [\text{g}] = \frac{rawAz - offsetAz}{16384} \tag{3}$$

$$\omega_x\ [\text{deg/s}] = \frac{rawGx - offsetGx}{131} \tag{4}$$

$$T\ [^\circ\text{C}] = \frac{rawT}{340} + 36.53 \tag{5}$$

10.4 Calibration Procedure

Calibration is a 3-phase process running approximately 5.5 seconds. The sensor must be placed flat and still on a level surface:

1. **Settle phase (500 ms):** Discard the first samples after power-on to allow the MEMS elements and DLPF to stabilise.
2. **Collect phase (5000 ms):** Sample at ≈200 Hz; accumulate ≥100 valid raw readings with a serial progress bar showing elapsed percentage.
3. **Compute offsets:**
   - $offset_{A_x} = \overline{A_x}$, $offset_{A_y} = \overline{A_y}$
   - $offset_{A_z} = \overline{A_z} - 16384$ (removes 1 g gravity from Z axis)
   - $offset_{G*} = \overline{G*}$ for each gyro axis

Calibration quality thresholds (verified after computation — if any fails, a warning is issued and calibration repeats):

Parameter	Threshold	Physical meaning
$\lvert a_x\rvert$, $\lvert a_y\rvert$ after cal	< 0.05 g	Sensor nearly level on X and Y
$\lvert a_z - 1\rvert$ after cal	< 0.05 g	Z axis sees exactly 1 g gravity
$\lvert\omega_x\rvert$, $\lvert\omega_y\rvert$, $\lvert\omega_z\rvert$	< 2.0 deg/s	Sensor is stationary
Samples collected	≥ 100	Statistical validity

Table 10.6: Calibration Quality Criteria

10.5 Relay Sequence Engine

The non-blocking sequence engine cycles through 10 steps using `millis()`-based timers (no `delay()`), ensuring IMU streaming continues uninterrupted:

Step	Action	Delay (ms)	Purpose
0	All relays OFF (initial state)	2000	Safety idle pause
1	Relay 1 ON	4000	Energise channel 1
2	Relay 2 ON	4000	Energise channel 2
3	Relay 3 ON	4000	Energise channel 3
4	Relay 4 ON (all ON)	4000	All channels energised
5	Relay 4 OFF	4000	De-energise channel 4
6	Relay 3 OFF	4000	De-energise channel 3
7	Relay 2 OFF	4000	De-energise channel 2
8	Relay 1 OFF (all OFF)	4000	All channels off
9	Increment cycle counter, restart	2000	Begin next cycle

Table 10.7: Relay Sequence Step Table

10.6 I2C Fault Recovery

After `MAX_CONSECUTIVE_FAILURES = 10` failed I2C transactions, the watchdog automatically: calls `Wire.end()`, waits 100 ms (allows bus lines to discharge), re-initialises I2C and all MPU6050 registers, runs a full re-calibration, and resumes streaming if it was active prior to the fault.

10.7 Local Serial Commands

Command	Action
`/status`	Print relay states, MPU status, uptime, packet stats
`/mpu`	Print one calibrated sensor reading
`/cal`	Trigger 5-second calibration
`/stream`	Toggle 5 Hz MPU streaming ON/OFF
`/r1`, `/r2`, `/r3`, `/r4`	Toggle individual relay
`/all`	All relays ON
`/off`	All relays OFF
`/seq`	Toggle sequence start/stop
any text	Send chat message to the Display node

Table 10.8: Serial Command Interface on Worker Node (115200 baud)

Chapter 11 : Display Node Firmware

File: `ESP32_Display/main/main.ino` (≈105 KB)

The Display firmware is the largest file in the project, managing the TFT rendering engine, touch input processing, ESP-NOW communication, 8-screen state machine, animated industrial scene, vibration data buffers, NVS storage, power management, and QR code display.

11.1 Display Specifications

Parameter	Value
Resolution	320 × 240 pixels
Colour depth	16-bit RGB565 (65,536 colours)
Display driver	ILI9341 (or compatible)
Driver library	TFT_eSPI (hardware SPI accelerated)
Touch IC	XPT2046 (12-bit ADC, SPI, VSPI bus)
Colour theme	Neon Industrial Dark
Frame rate	Up to 30 FPS (animation loop, 33 ms timer)

Table 11.9: Display Hardware Parameters

11.2 Screen Inventory

Enum	Name	Description
`SCR_HOME`	Home	Dashboard: status LED (red/green LIVE indicator), live IMU mini-readout, navigation icon grid
`SCR_CONTROL`	Control	Top-level control hub with relay overview and quick actions
`SCR_MANUAL`	Manual	Four relay cards with individual ON/OFF toggle buttons and state indicators
`SCR_AUTO`	Auto	Animated pneumatic cylinder industrial scene; sequence start/stop control
`SCR_INFO`	Info	Scrollable project info, component list, QR code linking to GitHub repository
`SCR_CONN`	Connection	MAC address display, peer pairing instructions, Wi-Fi channel credentials
`SCR_CAL`	Calibration	9-point touch calibration screen (auto-runs on first boot; re-runnable from menu)
`SCR_VIBRATION`	Vibration	Live 80-sample rolling line graph for X, Y, Z acceleration with per-axis colour coding

Table 11.10: Named Screens in the Display Firmware

11.3 Colour Palette — Neon Industrial Dark Theme

Macro	RGB565 Hex	Approximate Colour	UI Role
`BG_DEEP`	`0x0000`	Pure black	Background fill
`CYAN`	`0xB7FF`	Bright cyan	Primary accent, borders, headings
`CORAL`	`0x2A5F`	Deep coral/red	Error / OFF state
`NGREEN`	`0x87E0`	Neon green	OK / ON state
`AMBER`	`0x057F`	Amber/orange	Warning / intermediate
`PURPLE_V`	`0xD897`	Purple-violet	Vibration graph (Z-axis)
`METAL_WHITE`	`0xFFFF`	Pure white	Specular highlights on metallic UI elements
`DARK_PANEL`	`0x2104`	Very dark grey	Card panel backgrounds

Table 11.11: Colour Palette — Neon Industrial Dark Theme

11.4 Touch Calibration — 9-Point Affine Transform

On first boot (or when no calibration data is found in NVS Preferences), the device runs the 9-point calibration routine: nine crosshair targets are displayed one at a time across the screen; the user taps each target, providing 9 screen-coordinate / raw-ADC pairs.

These pairs are used to solve the 6-parameter affine transform via least-squares:

$$x_{\text{screen}} = a_0 \cdot x_{\text{raw}} + a_1 \cdot y_{\text{raw}} + a_2 \tag{6}$$

$$y_{\text{screen}} = a_3 \cdot x_{\text{raw}} + a_4 \cdot y_{\text{raw}} + a_5 \tag{7}$$

The six parameters $a_0 \ldots a_5$ are stored in NVS under the `tcal` namespace and persist across power cycles. This approach compensates for rotation, scaling, and shear errors inherent in resistive touch panels, achieving accuracy better than ± 2 pixels.

11.5 Vibration Screen

The Vibration Screen maintains three independent 80-sample ring buffers (`vibBufX`, `vibBufY`, `vibBufZ`) updated at 250 ms intervals by requesting `CMD_MPU_ONCE` from the Worker. Each buffer is plotted as a coloured rolling line graph inside a card panel:

- **X-axis:** Plotted in cyan
- **Y-axis:** Plotted in neon green
- **Z-axis:** Plotted in purple-violet

The graph auto-scales between $-2\,\text{g}$ and $+2\,\text{g}$. Numeric readouts of current values are displayed alongside the graph.

11.6 Auto Screen — Animated Industrial Scene

The Auto screen renders a simulated pneumatic actuator assembly for visual relay feedback:

- `cyl2`: Horizontal cylinder; rod extends from $x = 28$ to $x = 200$ (driven by Relay 1/2 state)
- `cyl3`: Vertical cylinder; rod descends from $y = 52$ to $y \approx 154$ (driven by Relay 3/4 state)
- Animation is state-machine driven with `CylState.pos` ranging 0.0–1.0 and speed 0.7 units/frame
- Frame timer: 33 ms (30 FPS); uses `millis()` non-blocking timer

11.7 QR Code — GitHub Repository

The Info screen includes a QR code generated at runtime using the bundled `qrcode` library (ECC level LOW, version 3) pointing to the project repository:

https://github.com/madebygyanesh/ARS_IIT

Scan to access full source code, circuit diagrams, and documentation

The QR image can be tapped on the device to enter full-screen view with a ZOOM button for easier scanning.

11.8 Power Management

```
#define SLEEP_TIMEOUT_MS  45000UL   // 45 seconds idle timeout

// In loop():
if (millis() - lastTouchMs > SLEEP_TIMEOUT_MS) {
    enterDeepSleep();
}

void enterDeepSleep() {
    tft.fillScreen(TFT_BLACK);
    // Configure GPIO 36 (XPT2046 IRQ, active LOW) as wake source
    esp_sleep_enable_ext0_wakeup((gpio_num_t)TOUCH_WAKEUP_PIN, 0);
    esp_deep_sleep_start();
    // Execution resumes from reset after wake
}
```

Listing 3: Deep-sleep with touch interrupt wakeup after 45 s idle

Chapter 12 : Project File Structure

The project repository is organised into the following directory structure:

```
Vibration_Measurement_Using_MPU6050/
|
+-- Circuit Diagram/
|   +-- ESP32_C3_MINI.jpg       Worker node wiring diagram
|   +-- ESP_32_Display.jpg      Display node wiring diagram
|
+-- ESP32_C3_Mini/
|   +-- C3_Worker_main/
|       +-- C3_Worker_main.ino  Worker firmware (~44 KB)
|                               - MPU6050 I2C driver
|                               - Calibration engine
|                               - ESP-NOW send/receive
|                               - Relay sequence engine
|                               - I2C fault recovery
|                               - Serial CLI handler
|
+-- ESP32_Display/
|   +-- main/
|   |   +-- main.ino            Display firmware (~105 KB)
|   |                           - TFT rendering engine
|   |                           - 8-screen state machine
|   |                           - Touch calibration
|   |                           - Vibration graph
|   |                           - Animated industrial scene
|   |                           - Deep-sleep management
|   |                           - NVS storage
|   +-- qrcode.cpp              QR code generation library
|   +-- qrcode.h                QR code header
|
+-- REQUIRED_Lib/
|   +-- TFT_eSPI.zip            TFT driver library (6.1 MB)
|   |                           - Supports ILI9341, ST7789, etc.
|   |                           - Hardware SPI acceleration
|   |                           - Custom fonts, sprites
|   +-- XPT2046_Touchscreen.zip Touchscreen library (81 KB)
|                               - SPI resistive touch driver
|                               - Raw ADC coordinate output
|
+-- README.md                   Project overview and setup guide
+-- Vibration_Measurement_Report.pdf  Technical report (this document)
```

Listing 4: Complete project file structure

12.1 Required Libraries — Installation

Library	File	Install Method
TFT_eSPI	`REQUIRED_Lib/TFT_eSPI.zip`	Sketch → Add .ZIP Library
XPT2046_Touchscreen	`REQUIRED_Lib/XPT2046_Touchscreen.zip`	Sketch → Add .ZIP Library

Table 12.12: External Libraries Requiring Installation

Library	**Usage**
`esp_now.h`	Peer-to-peer wireless packet exchange
`WiFi.h`	MAC address query, Wi-Fi mode setup
`esp_wifi.h`	Low-level channel configuration
`Wire.h`	I2C communication with MPU6050
`Preferences.h`	NVS persistent storage (touch calibration)
`esp_sleep.h`	Deep-sleep and wake source configuration
`SPI.h`	Hardware SPI for touchscreen (VSPI)
`math.h`	`sinf`, `cosf`, `fabsf` for cylinder animations

Table 12.13: Built-in Arduino/IDF Libraries (No Installation Required)

Chapter 13 : Software Architecture

13.1 System Block Diagram (Software)

```
+-----------------------------------------+
| ESP32-C3 Mini (Worker)                  |
|                                         |
| setup():                                |
|   initESPNOW() -> register peer MAC     |
|   initMPU6050() -> configure registers  |
|   calibrateMPU6050() -> 5-second cal    |
|   initRelays() -> all OFF (active LOW)  |
|                                         |
| loop() @ ~200 Hz:                       |
|   if newDataReceived -> handleReceived() |
|   runSequenceEngine()  [non-blocking]   |
|   if streaming & 200ms -> sendMPUData()  |
|   if failures >= 10   -> recoverMPU()   |
|   processSerialCommands()               |
|   delay(5)                              |
+-----------------------------------------+
         | DataPacket (193 B) via ESP-NOW
+-----------------------------------------+
| ESP32 (Display Node)                    |
|                                         |
| setup():                                |
|   initTFT() + calibrateTouch() (NVS)    |
|   initESPNOW() -> register worker MAC   |
|   showHomeScreen()                      |
|                                         |
| loop() @ ~30 FPS:                       |
|   if newDataRx -> handleReceivedPacket() |
|   updateActiveScreen()                  |
|   if SCR_VIBRATION & 250ms -> pollIMU() |
|   updateCylinderAnimation() [33ms tick]  |
|   handleTouchInput()                    |
|   if idle > 45s -> enterDeepSleep()     |
+-----------------------------------------+
```

Listing 5: Software architecture overview — both nodes

13.2 Key Source Code Excerpts

Worker: Calibrated IMU Read

```
bool readMPU6050() {
    if (!mpuFound) return false;
    int16_t rax, ray, raz, rgx, rgy, rgz, rtmp;
    if (!readMPU6050Raw(&rax,&ray,&raz,&rgx,&rgy,&rgz,&rtmp))
        return false;

    // Apply calibration offsets, convert to physical units
    accelX = ((float)rax - accelX_offset) / 16384.0f;  // +/-2g
    accelY = ((float)ray - accelY_offset) / 16384.0f;
```

```
    accelZ = ((float)raz - accelZ_offset) / 16384.0f;

    gyroX  = ((float)rgx - gyroX_offset)  / 131.0f;    // +/-250 deg/s
    gyroY  = ((float)rgy - gyroY_offset)  / 131.0f;
    gyroZ  = ((float)rgz - gyroZ_offset)  / 131.0f;

    mpuTemp = ((float)rtmp / 340.0f) + 36.53f;          // degrees C
    return true;
}
```

Listing 6: readMPU6050() with calibration offsets applied

Worker: ESP-NOW Callbacks

```
void onSent(const wifi_tx_info_t *info, esp_now_send_status_t status) {
    sendOk   = (status == ESP_NOW_SEND_SUCCESS);
    sendDone = true;
    if (sendOk) statSent++; else statFail++;
}

void onReceive(const esp_now_recv_info_t *recv_info,
               const uint8_t *data, int len) {
    if (len == sizeof(DataPacket)) {
        memcpy(&recvPkt, data, sizeof(DataPacket));
        newDataReceived = true;
        statRecv++;
    } else {
        Serial.printf("[WARN] Wrong packet size: got %d, expected %d\n",
                      len, (int)sizeof(DataPacket));
    }
}
```

Listing 7: ESP-NOW send and receive callbacks

Display: Relay Status Parsing

```
// In onDataRecv(), MSG_TYPE_RELAY_STATUS branch:
// text format: "R1:ON R2:OFF R3:OFF R4:OFF | SEQ:RUNNING | CAL:YES"
char *p = recvPkt.text;
for (int i = 0; i < 4; i++) {
    char *colon = strchr(p, ':');
    if (colon) {
        coil[i] = (strncmp(colon+1, "ON", 2) == 0);
        p = colon + 4;   // skip "ON " or "OFF"
    }
}
// Update relay card UI colours based on coil[] state
```

Listing 8: Parsing relay status text payload on Display node

Chapter 14 : Setup and Flashing Guide

14.1 Prerequisites

- **Arduino IDE 2.x** or **PlatformIO** (VS Code extension)
- **ESP32 board package** by Espressif Systems installed via Board Manager:
  - URL: `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
- Install via *Sketch* → *Include Library* → *Add .ZIP Library*:
  - `REQUIRED_Lib/TFT_eSPI.zip`
  - `REQUIRED_Lib/XPT2046_Touchscreen.zip`
- Edit `TFT_eSPI/User_Setup.h` to define your display driver and TFT pin assignments before compiling
- USB drivers for ESP32-C3 (CP2102 or CH340 depending on your board revision)

14.2 Step-by-Step Flashing

Step 1: Flash the Display Node first.

1. Open `ESP32_Display/main/main.ino`
2. Select board: **ESP32 Dev Module**; Partition: **Default 4MB**
3. Upload and open Serial Monitor at **115200 baud**
4. On first boot, complete the **9-point touch calibration** shown on screen
5. Navigate to the **Connection screen** — note the **"My MAC"** address displayed (e.g., `00:70:07:24:AC:4C`)

Step 2: Configure and flash the Worker Node.

1. Open `ESP32_C3_Mini/C3_Worker_main/C3_Worker_main.ino`
2. Edit the peer MAC address at the top of the file:

```
// Replace with the MAC address shown on the Display's Connection
   screen
uint8_t peerMacAddress[] = {0x00, 0x70, 0x07, 0x24, 0xAC, 0x4C};
```

3. Select board: **ESP32C3 Dev Module**; USB CDC on Boot: **Enabled**
4. Upload. On boot, the worker will auto-calibrate — keep the sensor **flat and still** for 5 seconds
5. Serial Monitor (115200 baud) will show calibration progress and final offset values

Step 3: Verify communication.

- The status LED on the Display's home screen top-bar should turn **green (LIVE)** within a few seconds
- Live IMU values should appear on the Home screen mini-readout
- The Worker serial output will show `[ACK]` and `[STATUS]` messages confirming reception
- Navigate to `SCR_VIBRATION` to see the live rolling waveform graph

14.3 Important Configuration Notes

Wi-Fi Channel Matching: Both nodes MUST use the same Wi-Fi channel (default: channel 1). If the Display Node also connects to a Wi-Fi router, the ESP-NOW channel must match the router's operating channel. Adjust `WIFI_CHANNEL` in both firmware files if needed.

Packet Size Assertion: A `static_assert` in the Worker firmware will produce a compile-time error if `DataPacket` is not exactly 193 bytes. Never add or remove fields from the struct without updating *both* firmware files simultaneously and re-flashing both nodes.

First Boot Calibration: During the 5-second calibration phase, the MPU6050 module must remain completely stationary on a flat, level surface. Any movement during this phase will produce incorrect offsets and must be re-triggered using the `/cal` serial command or `CMD_MPU_CALIBRATE` from the Display.

Chapter 15 : Advantages

1. Low-Cost Open-Source Platform

The entire system uses commodity components and open-source libraries with a total hardware cost under Rs. 1,650 — orders of magnitude cheaper than commercial wireless vibration analysers (which typically cost Rs. 50,000–5,00,000). Full source code is publicly available on GitHub.

2. Router-Free Wireless Communication

ESP-NOW operates completely without a Wi-Fi router or access point. This makes the system fully self-contained and deployable in remote industrial sites, basements, outdoor locations, or any environment where Wi-Fi infrastructure is unavailable.

3. Real-Time Low-Latency Streaming

The system achieves <10 ms ESP-NOW communication latency and 5 Hz continuous IMU data streaming — adequate for monitoring vibration signatures up to 2.5 Hz (Nyquist). The 44 Hz DLPF bandwidth preserves relevant industrial vibration content.

4. Production-Quality Fault Tolerance

Automatic I2C watchdog recovery, compile-time packet size validation, active-LOW relay safety defaults (relays default OFF on power loss), and multi-phase calibration with quality verification collectively make the system suitable for unattended operation.

5. Premium Industrial Touch-Screen HMI

The 8-screen Neon Industrial Dark themed UI with animated machinery, live waveform graphs, and relay control cards provides a professional user experience on a Rs. 350 display module.

6. Power Efficiency

Deep-sleep with 45-second idle timeout and touch-interrupt wakeup extends battery life significantly in portable or battery-powered deployments.

7. Dual-Node Architecture Benefits

Separating sensor acquisition (Worker) from display rendering (Display) ensures neither task interferes with the other. The Worker can stream sensor data at a guaranteed rate even while the Display is rendering complex animations.

8. Extensibility

The modular firmware design allows easy addition of more sensor channels, MQTT cloud logging via Wi-Fi, FFT frequency-domain analysis, Bluetooth data export, machine learning anomaly detection, and additional relay channels.

Chapter 16 : Applications

1. Industrial Predictive Maintenance

Monitoring vibration signatures of rotating machinery (motors, pumps, compressors, gearboxes, fans) to detect imbalance, bearing wear, misalignment, or resonance before catastrophic failure. The relay control capability enables automatic equipment shutdown when vibration thresholds are exceeded.

2. Structural Health Monitoring

Measuring vibration in civil structures — bridges, building floors, transmission towers, pipelines — to detect damage, fatigue cracks, resonance conditions, or seismic activity. The wireless architecture eliminates cable routing in large structures.

3. Academic and Educational Demonstrations

Providing a low-cost, fully open-source platform for teaching embedded systems design, MEMS sensor calibration, ESP-NOW wireless protocol, real-time signal processing, and industrial HMI development to engineering students.

4. Automated Actuator Test Rigs

The relay sequence engine enables automated cycling of pneumatic valves, solenoids, or motor starters in quality control and endurance testing applications, as visualised by the animated industrial scene on the Auto screen.

5. Agricultural and Environmental Monitoring

Detecting ground vibrations from seismic activity, heavy machinery near crops, or pipeline leaks in remote agricultural or field locations where infrastructure Wi-Fi is unavailable.

6. Robotics and Motion Analysis

Monitoring vibration and motion in robotic joints, end-effectors, or mobile platforms to improve trajectory control algorithms and detect mechanical wear or loosening.

7. Healthcare — Tremor Monitoring

Measuring hand or limb tremor in patients with neurological conditions (Parkinson's disease, essential tremor) as a low-cost wearable diagnostic aid for clinicians.

8. Smart Infrastructure and IoT Integration

Integrating with cloud IoT platforms (AWS IoT, Google Cloud IoT, ThingSpeak) via the ESP32's Wi-Fi capability for remote monitoring dashboards, historical trending, and automated alert notifications.

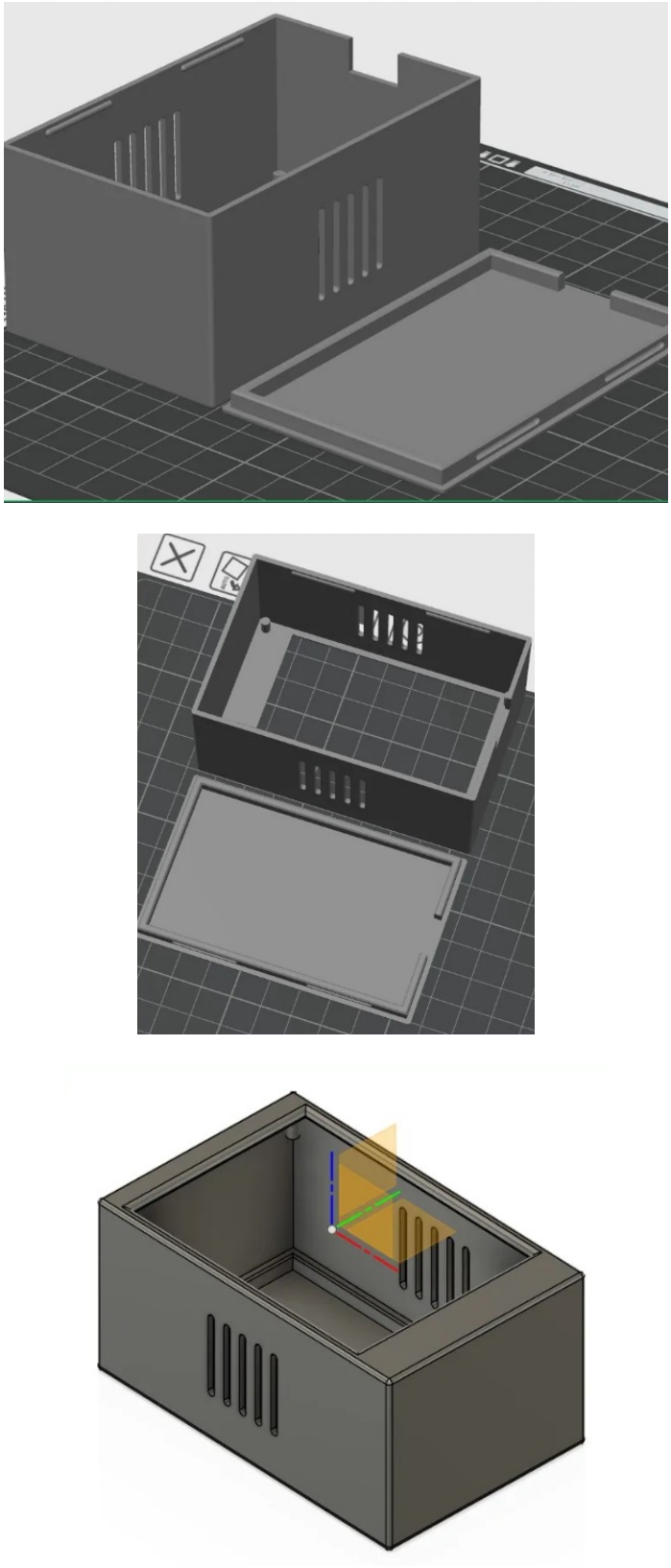

Figure 16.3: 3D Model View

Chapter 17 : Overview

The **Vibration Measurement System Using MPU6050 & Dual ESP32 Architecture** implements a complete wireless embedded platform for industrial vibration sensing and relay automation. The following table summarises all key system parameters:

Parameter	**Value**
IMU sensor	MPU6050 / MPU6500 (6-axis MEMS)
Accelerometer FSR	± 2 g (16384 LSB/g)
Gyroscope FSR	± 250 deg/s (131 LSB/(deg/s))
ADC resolution	16-bit
DLPF bandwidth	≈ 44 Hz
Internal sample rate	≈ 200 Hz
Wireless data streaming rate	5 Hz
Communication protocol	ESP-NOW (2.4 GHz)
DataPacket size	193 bytes (compile-time verified)
Communication latency	<10 ms typical
Relay channels	4 (active-LOW)
Relay sequence steps	10 (round-robin, configurable timing)
Display resolution	320×240 pixels, 16-bit RGB565
Display driver	ILI9341 via TFT_eSPI
Touch controller	XPT2046 (VSPI bus)
Touch calibration	9-point affine transform (NVS-persisted)
UI screens	8 named screens
Animation frame rate	30 FPS (33 ms timer)
Vibration graph samples	80-sample rolling buffer (3 axes)
Deep-sleep idle timeout	45 seconds
Wake source	GPIO 36 (touch IRQ, active LOW)
Calibration duration	≈ 5.5 seconds
I2C fault recovery threshold	10 consecutive failures
Total hardware cost	$\approx$Rs. 1,634

Table 17.14: Complete System Specifications Summary

Chapter 18 : Conclusion

The project **Vibration Measurement System Using MPU6050 & Dual ESP32 Architecture** successfully demonstrates a robust, production-quality embedded system that combines MEMS sensor integration, peer-to-peer wireless communication, relay automation, and an industrial-grade touch-screen HMI into a single affordable platform costing under Rs. 1,650.

The **dual-node architecture** proves highly effective: the Worker Node runs time-critical sensor acquisition and relay actuation with no display overhead, while the Display Node manages complex UI rendering and user interaction without impacting sensor timing. The two nodes remain perfectly synchronised via the 193-byte ESP-NOW `DataPacket`, whose compile-time size assertion prevents firmware integration errors.

Engineering quality is evident throughout: the multi-phase IMU calibration with threshold verification ensures accurate physical units; the I2C fault recovery watchdog maintains operation in electrically noisy environments; the active-LOW relay defaults ensure safe power-up states; and the NVS-persisted touch calibration eliminates recalibration after every power cycle.

The **8-screen Neon Industrial Dark HMI** — featuring live vibration waveforms, animated pneumatic cylinder scenes, relay control cards, and an embedded QR code — demonstrates that professional-quality user interfaces can be achieved on low-cost embedded hardware.

Future enhancements could include:

- Machine learning-based vibration anomaly detection using TensorFlow Lite for Microcontrollers
- FFT frequency-domain analysis for rotating machinery fault classification
- MQTT over Wi-Fi for cloud data logging and remote monitoring dashboards
- Additional IMU channels for multi-point structural health monitoring
- IP-rated 3D-printed industrial enclosure for field deployment
- Over-the-air (OTA) firmware update support via ESP32's Wi-Fi

Overall, this project exemplifies how ESP32-based IoT systems can be engineered to production standards while remaining accessible, affordable, and fully open source.

Full Source Code: `https://github.com/madebygyanesh/ARS_IIT`

Chapter 19 : References

1. Espressif Systems, "ESP-NOW — User Guide," 2023. [Online]. Available: `https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html`

2. InvenSense Inc., "MPU-6050 Product Specification," Rev. 3.4, 2013. [Online]. Available: `https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/`

3. InvenSense Inc., "MPU-6500 Product Specification," Rev. 1.3, 2014. [Online]. Available: `https://invensense.tdk.com/`

4. Espressif Systems, "ESP32-C3 Series Datasheet," Rev. 1.6, 2023. [Online]. Available: `https://www.espressif.com/`

5. Espressif Systems, "ESP32-WROOM-32D Datasheet," Rev. 3.2, 2022. [Online]. Available: `https://www.espressif.com/`

6. Bodmer, "TFT_eSPI — TFT/LCD Driver for ESP32," GitHub, 2024. [Online]. Available: `https://github.com/Bodmer/TFT_eSPI`

7. P. Stoffregen, "XPT2046_Touchscreen Library," GitHub, 2024. [Online]. Available: `https://github.com/PaulStoffregen/XPT2046_Touchscreen`

8. Arduino Community, "ESP32 Arduino Core Documentation," 2024. [Online]. Available: `https://docs.espressif.com/projects/arduino-esp32/`

9. G. Maurya, "ARS_IIT — Vibration Measurement Using MPU6050," GitHub, March 2026. [Online]. Available: `https://github.com/madebygyanesh/ARS_IIT`

10. C. Lynch and J. Loh, "A Summary Review of Wireless Sensors and Sensor Networks for Structural Health Monitoring," *Shock and Vibration Digest*, vol. 38, no. 2, pp. 91–128, 2006.

11. C. Farrar and K. Worden, "An Introduction to Structural Health Monitoring," *Philosophical Transactions of the Royal Society A*, vol. 365, no. 1851, pp. 303–315, 2007.

12. S. O. H. Madgwick, "An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays," Dept. of Mechanical Engineering, University of Bristol, Technical Report, 2010.

13. P. Kumar and R. P. S. S. Chandel, "Air Mouse Using an MPU-6050 Sensor," in *Proc. ICACCI*, 2016, pp. 1294–1298.

14. D. Titterton and J. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed. Reston, VA, USA: AIAA, 2004.

15. IEEE Standards Association, "IEEE 802.11-2020 Standard for Information Technology — Wireless LAN MAC and PHY Specifications," 2020.

Chapter 20 : Acknowledgments

We would like to express our sincere gratitude to all those who supported and guided us during the completion of this project on **Vibration Measurement System Using MPU6050 & Dual ESP32 Architecture**.

First and foremost, we are deeply thankful to **MR. Imran Rajwani, Mrs. Shaikh Naema**, our project guide, for her invaluable guidance, continuous encouragement, and constructive suggestions throughout the development process. Her expertise in embedded systems and IoT was instrumental in resolving technical challenges and refining the system design.

We are grateful to **Anjuman-I-Islam's Abdul Razzak Kalsekar Polytechnic, Panvel**, and the faculty members of the **Department of Automation & Robotics** for providing the laboratory resources, electronic components, soldering equipment, and technical support required for the successful implementation of this project.

We acknowledge the invaluable contributions of the open-source community — in particular Bodmer (TFT_eSPI library), Paul Stoffregen (XPT2046_Touchscreen library), and the Espressif Systems team (ESP32 Arduino Core and ESP-IDF documentation) — whose tools and libraries made rapid development of this system possible.

We also thank the **AI coding assistants** — Claude Sonnet, Gemini AI, GLM 4.5, and GitHub Copilot — which assisted in writing, debugging, and optimising the firmware code during development.

Finally, we thank our families and colleagues for their patience and encouragement throughout the duration of this project. This work has provided us with valuable practical experience in embedded systems design, wireless communication protocols, MEMS sensor integration, industrial HMI development, and IoT engineering.

Gyanesh Maurya

Mohammad Ibaad Nadkar

Md. Ayan Ansari

Huzaifa Adi

Department of Automation & Robotics
Anjuman-I-Islam's Abdul Razzak Kalsekar Polytechnic
Panvel, Maharashtra, India
March 2026